

To get set up on sherlock, visit <https://www.sherlock.stanford.edu/> and request an account

The sentieon pipeline is controlled by **bash** scripts. The **submit_all.sh** script queues up **jobs using slurm**. Any command you could run on a linux or macos terminal, you can run in a bash script. Bash scripts run commands in parallel and do not wait for the execution of any command in its body to finish before continuing. Bash scripts have access to any files on Sherlock you would have access to, and don't to any you wouldn't unless somehow modified in the body of the script itself. Bash scripts start running using your current directory as its working directory, but the submit_all.sh script doesn't use any relative directories for its run.

As of writing this documentation, you can find the pipeline on sherlock at:
/oak/stanford/groups/cgawad/Scripts/Sentieon_Pipeline

Other important scripts are found in:
/oak/stanford/groups/cgawad/Scripts

Reference files that might be important are stored in:
/oak/stanford/groups/cgawad/Reference_Files

Preexisting environments can be found at:
/home/groups/cgawad

Note that the **WGS_WES_Pipeline_2.0** was the predecessor to this pipeline, and can be used to reference commands that formed the basis for the sentieon pipeline

I recommended looking at the following tutorial on bash scripting before getting started:
<https://www.freecodecamp.org/news/bash-scripting-tutorial-linux-shell-script-and-command-line-for-beginners/>

I also recommend the following tutorials on how sherlock work:
<https://www.sherlock.stanford.edu/docs/user-guide/running-jobs/#persistent-jobid>
<https://www.sherlock.stanford.edu/docs/software/modules/#module-usage>

Temporary working pipeline directory

- The pipeline creates a working directory with the name you set in **--project** in your **\$SCRATCH** directory, a temporary directory with faster i/o on sherlock (try running **echo \$SCRATCH** if you are confused on where this is). Files are

generated there when you run the pipeline, then copied to the **--final_dir** specified directory, which should be on \$OAK, when the entire pipeline finishes, assuming nothing went wrong. You can look here for intermediary files while the job is running

std_err_out_files

- In both the working directory and final directory, you should always see a folder called **std_err_out_files**, which contains many log files, some with .out, some with .err, and with a hopefully descriptive name of what part of the pipeline was being executed with those log files
- The **.out** file contains the text from the stdout stream from that job run in the pipeline
- The **.err** file contains the stderr stream from that job run in the pipeline
- Usually **.err** will have more useful information explaining if and why a job went wrong, but **.out** files can be useful as well

set -x

- set -x, when run in a bash script, will stream the values of variable declarations and other information to the stdout. Some files have this activated by default and others don't. This makes the output very wordy, but can help catch when variables are getting null values or the wrong value assigned to them

Python and R scripts

- The pipeline also uses python and R scripts, located within the pipeline repo.

Modules not installed by the sherlock team

- You do not have to use modules installed by the sherlock team. As long as you don't need root privileges to install a binary, a package, or other piece of software, you can download or upload them to a directory you have permission to on sherlock and either **source** the binary or software directly or create an **environment** to use it
- A commonly activated conda environment containing python and R scripts used in the python can be activated in the following way:

Unset

```
source /home/groups/cgawad/miniconda3/etc/profile.d/conda.sh
```

```
conda deactivate
```

```
conda activate update_scan2
```

- This should show you an example of how you can activate an environment containing files not installed by the sherlock team. You can then use conda install commands to install whatever packages you want through conda.

WES vs WGS

- Sequencing data may be either **exome capture** or **whole genome sequencing**. Be sure to include the **--exome** argument in your command if it was an exome sequencing run. There were multiple exome panels used by the lab for whole exome sequencing experiments. To set the pipeline to run panel version two, add the following argument: **--version 2** (be sure to include the space and also to include the **--exome** argument as well!). If you want to run version 1, add **--version 1** instead. Here is an example command:

Unset

```
sh /oak/stanford/groups/cgawad/Scripts/Sentieon_Pipeline/submit_all.sh
--final_dir/oak/stanford/groups/cgawad/Nonmalignant_Tissue/Seizure_EEG_Probes/033
123_EEG_Pt01_exome_nextseq_sentieon
--fastq_dir/oak/stanford/groups/cgawad/Illumina_Data/NextSeq2000/2022_06_06_VascM
_EEG/Patient_1 --normal_sample_nameSeizures_Pt1_Bulk_PBMC_S23 --number_threads 16
--projectEEG_Pt01_exome_nextseq_sentieon --exome --version 2
```

The "--final_dir" is the folder you want your output files to be placed into, note that the pipeline will put all of the files in that folder so be sure you are read for a bunch of files to be copied into it. The "--fastq_dir" should point to the directory containing fastqs. The "--normal_sample_name" should be the sample name on the sample sheet used during demultiplexing/fastq generation. This should be everything before "_L001_R1_001.fastq.gz" in your fastq file name, i.e. if the file name is "Lung_Pt1_NE_Adjacent_Bulk_S4_L001_R1_001.fastq.gz" the bulk name will

beLung_Pt1_NE_Adjacent_Bulk_S4. Your project name should be specific to the run you are doing, a good template is: project_patientID_libraryType_sentieon

Here is a template, replace everything in **brackets** (including the brackets) with your file, and remember to change options like --exome and --version 1 if applicable to you:

Unset

```
sh /oak/stanford/groups/cgawad/Scripts/Sentieon_Pipeline/submit_all.sh  
--final_dir [final_dir] --fastq_dir [fastq_dir] --number_threads 16 --project  
[project_name] --exome --version 1
```

If you have any questions about running the pipeline don't hesitate to ask me! You may want to shoot me or Chuck an email once you've started running commands.

Here is a brief explanation of some of the new files outputted:

01_all_germline_snp_indel.tsv<- contains all germline snps indels (header off, mainly for grepping)

01_germline_known_pathogenic.tsv<- contains all known pathogenic germline snps/indels (header off, mainly for grepping)

01_(snp/indel)_germline_known_pathogenic.tsv<- same as above but split into two files so header isn't off

01_all_variants.tsv<- contains all variants, header is off

01_all_somatic_snvs.tsv<- contains all somatic snv calls

01_all_somatic_indels.tsv<- contains all somatic indel calls

01_final_clonal_snp_indel_calls.tsv<- contains all clonal snvs/indels, header is off

01_final_clonal_indel_calls.tsv<- contains all clonal indels

01_final_clonal_somatic_snvs.tsv<- contains all clonal snvs

01_functional_clonal_somatic_(snvs/indels).tsv<- contains all snvs/indels that are non-synonymous, stopgain/loss, splicing, exonic deletions or insertions

01_final_non_clonal_somatic_calls.tsv<- contains all nonclonal (only in one cell) snv calls

01_combined_stacked_mutsig_plot.pdf<- contains combination of mutation signature stacked barplots for all cells

01_individual_stacked_signatures.pdf<- contains mutation signature stacked barplots for each cell individually, merged into one pdf